

4.1 Problema 1: Maximização de Lucro (Alocação de Frota)

Contexto: Analisando a tabela **Entregadores** da base de dados da LogiPrime, operamos com dois modais principais na coluna **Veículo**: **Motorcycle** (Motocicleta) e **Scooter**. A gerência precisa definir o limite diário de despachos para cada tipo de veículo visando maximizar o lucro, respeitando a jornada máxima de trabalho e regras de segurança no trânsito.

- **Motorcycle (x1):** Entregas mais rápidas, geram um lucro líquido de R\$ 25,00 por pedido. Consome em média 0,5 horas do entregador.
- **Scooter (x2):** Entregas mais lentas e locais, geram lucro de R\$ 15,00. Consomem em média 0,75 horas.

A operação dispõe de um máximo de **150 horas** totais de agentes por dia. Além disso, por diretrizes de segurança, o limite máximo de envios por motos (*Motorcycle*) não pode ultrapassar **200 entregas** diárias.

Variáveis de Decisão:

- x1: Quantidade de entregas alocadas para Motorcycle.
- x2: Quantidade de entregas alocadas para Scooter.

Função Objetivo (Maximizar o Lucro): $\text{Max } Z = 25x1 + 15x2$

Restrições Técnicas:

- Horas Disponíveis: $0.5x1 + 0.75x2 \leq 150$
- Limite de Segurança (Motos): $x1 \leq 200$
- Lógica: $x1, x2 \geq 0$ (Inteiros)

Código Python (Google Colab / **pulp**):

```
from pulp import LpMaximize, LpProblem, LpVariable

model = LpProblem(name="Max_Lucro_LogiPrime", sense=LpMaximize)

x1 = LpVariable(name="Qtd_Motorcycle", lowBound=0, cat="Integer")
x2 = LpVariable(name="Qtd_Scooter", lowBound=0, cat="Integer")

model += 25 * x1 + 15 * x2, "Lucro_Total"

model += (0.5 * x1 + 0.75 * x2 <= 150, "Restricao_Horas_Agentes")
model += (x1 <= 200, "Limite_Seguranca_Moto")

model.solve()

print(f"Status: {model.status}")
print(f"Entregas via Motorcycle: {x1.varValue}")
print(f"Entregas via Scooter: {x2.varValue}")
print(f"Lucro Máximo: R$ {model.objective.value():.2f}")
```

Análise de Cenário (What-If):

- **Cenário de Crise:** E se a condição climática (coluna **Weather** = *Stormy*) piorar e a gerência decidir reduzir drasticamente o uso de motos por risco de acidentes, caindo o limite de segurança de 200 para apenas 80 entregas?
 - **Resultado Obtido:** Alterando a restrição no código para **x1 <= 80** e rodando novamente, o algoritmo adapta a operação instantaneamente. Em vez de focar nas motos, o modelo redireciona a força de trabalho e sugere realizar 80 entregas de moto e 146 de Scooter. O lucro cai (de R\$ 5.990 para R\$ 4.190), mas a matemática prova qual é a melhor alocação possível para não deixar os pacotes parados no galpão, garantindo a continuidade do negócio durante a tempestade.
-



4.2 Minimização de Custos (Distribuição Galpão x Destino)

Contexto: Utilizando as tabelas **Galpões** e **Entregas**, a LogiPrime precisa enviar pacotes de 2 centros de distribuição (Origens: São Paulo e Campinas) para 3 cidades de destino (Santos, São José dos Campos e Ribeirão Preto) com o menor custo de frete possível, resolvendo o problema de ineficiência de rotas levantado no projeto.

- **Oferta (Origem):** São Paulo (600 pacotes disponíveis), Campinas (450 pacotes).
- **Demanda (Destino):** Santos precisa de 400, S.J. dos Campos precisa de 350, Ribeirão Preto precisa de 300.
- **Custos de Frete (R\$ por pacote):** SP para (Santos=5, S.J.Campos=8, Ribeirão=15). Campinas para (Santos=12, S.J.Campos=10, Ribeirão=7).

Variáveis de Decisão:

- x_{ij} : Quantidade de pacotes enviada do Galpão de origem (i) para a Cidade de destino (j).

Função Objetivo (Minimizar Custo): $\text{Min } Z = 5x_{11} + 8x_{12} + 15x_{13} + 12x_{21} + 10x_{22} + 7x_{23}$

Restrições Técnicas:

- O total enviado de cada galpão não pode ultrapassar sua Oferta.
- O total recebido em cada cidade deve ser exatamente igual à sua Demanda.

Código Python (Google Colab / **pulp**):

```
from pulp import LpMinimize, LpProblem, LpVariable, lpSum

galpoes = ['Sao_Paulo', 'Campinas']

destinos = ['Santos', 'SJ_Campos', 'Ribeirao_Preto']

oferta = {'Sao_Paulo': 600, 'Campinas': 450}

demanda = {'Santos': 400, 'SJ_Campos': 350, 'Ribeirao_Preto': 300}

custos = {
    'Sao_Paulo': {'Santos': 5, 'SJ_Campos': 8, 'Ribeirao_Preto': 15},
    'Campinas': {'Santos': 12, 'SJ_Campos': 10, 'Ribeirao_Preto': 7}
}

model = LpProblem(name="Min_Custo_LogiPrime", sense=LpMinimize)

rotas = [(i, j) for i in galpoes for j in destinos]

x = LpVariable.dicts("Rota", rotas, lowBound=0, cat="Integer")

model += lpSum([custos[i][j] * x[(i, j)] for i in galpoes for j in destinos])

for i in galpoes:
    model += lpSum([x[(i, j)] for j in destinos]) <= oferta[i]

for j in destinos:
    model += lpSum([x[(i, j)] for i in galpoes]) == demanda[j]

model.solve()

print(f"Status: {model.status}")

for i in galpoes:
    for j in destinos:
        if x[(i, j)].varValue > 0:
            print(f"De {i} para {j}: {x[(i, j)].varValue} pacotes")
```

Análise de Cenário (What-If):

- **Cenário de Crise:** E se a base apontar trânsito severo (coluna **Traffic** = *Jam*) na rodovia que liga Campinas a Ribeirão Preto, aumentando o custo dessa rota de R\$ 7,00 para R\$ 18,00 por pacote?
 - **Resultado Obtido:** Modificando o dicionário de custos no código, o Python realiza o desvio inteligente da carga. Ele para de utilizar a rota congestionada e redistribui o fluxo, forçando o galpão de São Paulo a atender Ribeirão Preto. Isso evita que a frota fique presa no engarrafamento e freia o aumento exponencial dos custos operacionais daquele dia.
-

4.3 Análise Crítica Exigida (Otimização)

Relatório Técnico - Validação da Modelagem:

Sim, os resultados matemáticos obtidos no Python para a Maximização e Minimização fazem total sentido para a realidade de negócios da LogiPrime. Como mapeado no escopo do projeto, a empresa sofria com ineficiência no envio de pedidos, baseando-se apenas na ordem de chegada. Através da Pesquisa Operacional aplicada à nossa base de dados (cruzando variáveis como tempo de entrega, tipo de veículo e polos logísticos), conseguimos provar que a alocação de frota deve obedecer limites de tempo e segurança para garantir lucro líquido, e que a distribuição entre Centros de Distribuição e Cidades precisa ser otimizada via algoritmo, reduzindo o desperdício de combustível e tempo ocioso.

Além disso, a Análise de Cenário (*What-If*) mostrou-se uma ferramenta indispensável para a gestão de riscos e tomada de decisão segura no dia a dia. Ao simularmos imprevistos extraídos das nossas próprias tabelas — como clima ruim (*Stormy*) e trânsito severo (*Jam*) —, os algoritmos recalcularam as rotas e os limites de frota instantaneamente. Isso permite que a gerência da LogiPrime deixe de agir de forma reativa diante de crises; com os scripts desenvolvidos, a empresa pode inserir as novas variáveis pela manhã e obter um plano de contingência matemático que minimiza perdas financeiras e garante o cumprimento das demandas, trazendo previsibilidade e segurança operacional.